

ELT Portfolio Intelligence Agent

Agentic AI for Executive Project, Financial, Risk & Trend Intelligence

Technical Project Documentation | Version 1.0 | September 2026

1. Executive Summary

The ELT Portfolio Intelligence Agent is an Agentic AI solution for Executive Leadership Teams. It combines Jira project-delivery data with financial performance, persistent historical context and deterministic risk analysis to provide a single executive view of portfolio health. The solution is designed to answer leadership questions such as which projects need attention, what has been blocked for multiple sprints, which projects are consuming budget faster than they are progressing, and what changed since the previous reporting period.

The application is more than a chatbot: LangGraph coordinates a multi-step workflow, Mem0 provides persistent context, Python performs authoritative calculations, validation gates check evidence, and Streamlit presents dashboards and conversational analysis.

2. Business Objectives

- Unify delivery and financial health for active projects.
- Identify persistent blockers, aging work and delivery deterioration.
- Track approved funding, actual/committed spend, forecast and remaining funds.
- Compare current project state with previous weeks and sprints.
- Generate evidence-backed GREEN / AMBER / RED risk status.
- Provide concise executive recommendations with confidence and provenance.

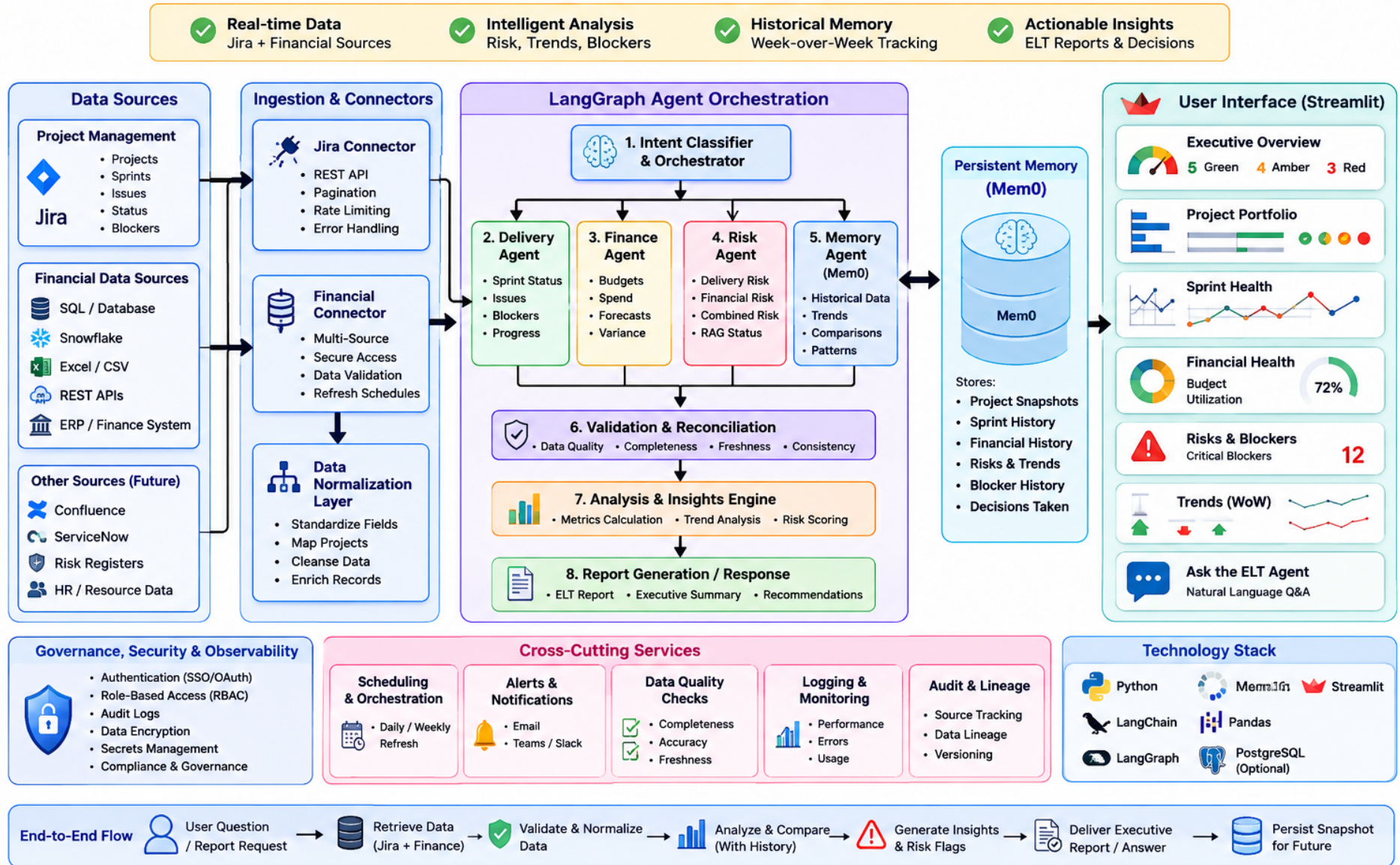
3. Agentic AI Architecture

The system is agentic because it can interpret an ELT question, select the required data and analysis path, retrieve authoritative source data, consult historical memory, calculate metrics, assess risk, validate evidence and generate an executive response while maintaining workflow state.

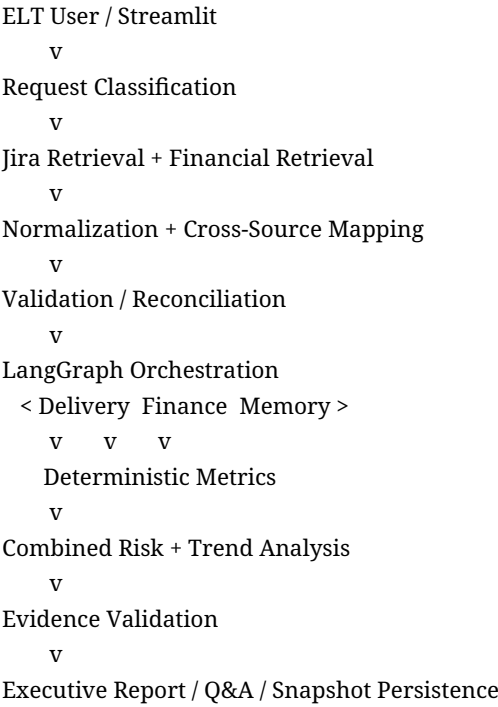
Figure 1. ELT Portfolio Intelligence Agent architecture.

ELT Portfolio Intelligence Agent - Architecture Diagram

ELT Project & Financial Intelligence Agent Architecture Overview



4. End-to-End Flow



5. Major Components

Component	Purpose
Jira Connector	Retrieves projects, sprints, issues, blockers and issue history.
Financial Connector	Provides a replaceable abstraction for CSV, SQL, Snowflake, Oracle or REST financial sources.
Cross-Source Mapping	Maps Jira project keys and finance IDs to a canonical project ID without guessing.
Pydantic Models	Strongly types Project, Sprint, JiraIssue, FinancialRecord, ProjectSnapshot and Risk.
LangGraph	Coordinates the stateful multi-agent workflow.
Delivery Agent	Analyzes sprint health, blockers, carryover and delivery risk.
Finance Agent	Analyzes budget, spend, forecast, remaining funds and financial risk.
Memory Agent / Mem0	Stores/retrieves normalized historical project facts and snapshots.
Risk Agent	Combines delivery and financial signals into overall RAG status.
Validation Agent	Checks freshness, completeness, mappings, reconciliation, evidence and confidence.
Report Agent	Creates executive summaries and recommendations from validated facts.
Streamlit	Provides executive dashboards, audit views, filters and conversational Q&A.

6. LangGraph Workflow

START

-> classify_request
-> fetch_delivery_data
-> validate_delivery_data
-> fetch_financial_data
-> validate_financial_data
-> retrieve_historical_memory
-> calculate_metrics
-> analyze_delivery_risk
-> analyze_financial_risk
-> analyze_cross_domain_risk
-> validate_findings
-> generate_response
-> persist_snapshot
-> END

Graph state carries the user question, project filters, Jira data, financial data, historical context, calculated metrics, risks, validation results, provenance and final answer. Nodes are designed not to silently overwrite facts produced by source systems.

7. Data Model

Model	Key Information
Project	Identity, ownership, schedule, budget, spend, progress and risk.
Sprint	Dates, committed/completed points, completion and carryover.
JiraIssue	Issue identity, status, priority, assignee, story points, blocker age and dependencies.
FinancialRecord	Approved budget, actual, committed, forecast, remaining and report date.
ProjectSnapshot	Historical sprint/financial metrics, RAG, blockers and source timestamp.
Risk	Category, severity, evidence, recommended action and detection time.
SourceProvenance	Source system, source record ID and retrieval timestamp.

8. Deterministic Metrics and Risk

Authoritative calculations are performed in Python rather than by the LLM:

```
remaining_budget = approved_budget - actual_spend - committed_spend  
budget_consumption_pct = (actual_spend / approved_budget) * 100  
forecast_variance = approved_budget - forecast_spend  
spend_to_progress_ratio = budget_consumption_pct / delivery_progress_pct
```

Delivery risk uses sprint completion, blocker age, multi-sprint persistence, dependencies and schedule delay. Financial risk uses budget consumption, forecast overrun, remaining funding and spend-versus-progress. Configurable rules combine these into GREEN, AMBER or RED.

9. Memory and Week-over-Week Intelligence

Mem0 acts as the persistent contextual memory layer. The system stores normalized historical facts and ProjectSnapshot records rather than raw Jira/finance payloads. Historical statements require real stored evidence. For example, an issue is considered stuck across three sprints only when the same Jira issue key is observed as blocked across those sprints or verified through Jira history.

- What has been stuck for more than one sprint?
- Which projects remained RED for multiple weeks?
- Which blockers appeared last week and still exist?
- Which projects are deteriorating?
- Which projects are spending faster than they are progressing?

10. Accuracy and Explainability

Accuracy Control	Purpose
Source provenance	Trace material facts to source records and retrieval time.
Freshness checks	Identify stale Jira, finance or historical data.
Completeness checks	Surface missing mappings, budgets, sprints or reporting periods.
Financial reconciliation	Flag inconsistent financial records.
Deterministic calculations	Prevent LLM-generated authoritative numbers.
Historical verification	Require multiple snapshots/history for persistence claims.
Evidence requirement	Require supporting facts for RED/high-risk claims.
Confidence rating	Return HIGH, MEDIUM or LOW confidence.
No-hallucination policy	Use UNKNOWN / NOT AVAILABLE / NOT MAPPED instead of guessing.
Final validation gate	Validate mappings, evidence, history and freshness before response.

11. Streamlit Executive Experience

The repository implements an executive Streamlit experience with the following pages:

- Executive Overview
- Project Portfolio
- Sprint Health
- Financial Health
- Risk & Blockers
- Trends

- Ask the ELT Agent
- Data Quality / Audit

The UI is backed by the same LangGraph pipeline used by the notebooks, allowing dashboards and conversational answers to share the same validated data and risk logic.

12. Repository Structure

```
elt-portfolio-agent/  
├─ app/           # Streamlit UI and backend  
├─ config/        # risk, project and status mappings  
├─ data/sample/   # bundled sample Jira/finance data  
├─ docs/          # worked ELT report  
├─ notebooks/     # phased development/validation notebooks  
├─ src/  
│   ├── agents/  
│   ├── connectors/  
│   ├── graph/  
│   ├── models/  
│   ├── services/  
│   └─ utils/  
├─ tests/  
├─ .env.example  
├─ architecture.md  
├─ README.md  
└─ requirements.txt
```

13. Technology Stack

Technology	Role
Python	Core application, connectors, metrics, risk and validation.
LangChain	LLM/tool integration.
LangGraph	Stateful agent orchestration.
Mem0	Persistent memory.
Pydantic	Typed domain models.
Streamlit	Executive UI and Q&A.
Plotly	Dashboard visualizations.
Jira REST API / Requests	Delivery source integration.
Pandas / Jupyter	Data handling, development and analysis.
PyYAML	Configurable rules and mappings.
Pytest	Automated testing.

14. Sample Data and Validation Scenarios

The repository includes bundled sample data so notebooks and the dashboard can run without external credentials. Its architecture documentation describes hundreds of Jira issues across multiple projects,

monthly financial records, blocked issues, schedule-stressed projects, financially stressed projects, stale-data scenarios, a Jira-only project with missing finance mapping, and a deliberate source inconsistency for validation testing.

- Healthy/low-risk delivery behavior
- Low sprint completion / schedule stress
- High budget consumption / forecast stress
- Persistent blockers
- Missing project-to-finance mapping
- Stale source data
- Source data-quality inconsistency
- Baseline behavior before enough history exists

15. Installation and Execution

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
# Optional for live Jira / Mem0 / LLM
cp .env.example .env
```

```
# Development notebooks
jupyter notebook notebooks/01_environment_setup.ipynb
```

```
# Executive dashboard
streamlit run app/streamlit_app.py
```

The project README states that the notebooks can run end-to-end against bundled sample data without external credentials.

16. Testing, Security and Observability

- Unit tests cover financial calculations, sprint metrics, blocker persistence, risk classification, mappings, trends and failure conditions.
- Secrets are externalized through environment variables and must never be committed or logged.
- Structured observability records graph node execution, errors, validation outcomes and latency while redacting credential-shaped fields.
- Source outages are designed to degrade affected values to UNKNOWN rather than crash the graph or fabricate data.
- Future enterprise deployment can add SSO, RBAC, managed secrets and centralized audit logging.

17. Current Implementation Status

The repository README identifies the solution as Phase 12 of 12 complete. It reports implemented Jira and financial connectors, cross-source unification, delivery/financial/combined risk engines, Mem0

historical memory, full LangGraph orchestration, ELT report generation, Streamlit dashboards, a consolidated accuracy audit, structured observability logging and production hardening for total Jira or financial source outages.

Note: architecture.md retains its original Phase 1 framing in the document title/scope, while the current README reflects the completed implementation. This documentation therefore uses the current repository status as the implementation baseline and the architecture file as the design reference.

18. Future Enhancements

- Connect production Jira Cloud and enterprise financial systems.
- Add Confluence/SharePoint RAG for charters, decisions and meeting notes.
- Add enterprise SSO/RBAC and managed secrets.
- Schedule weekly executive report distribution and alerts.
- Add forecasting and predictive early-warning models.
- Add human approval for high-impact escalations/recommendations.
- Integrate enterprise observability and audit platforms.

19. Conclusion

The ELT Portfolio Intelligence Agent moves executive reporting from static status aggregation toward proactive decision intelligence. Its core strength is the controlled combination of delivery, finance and history: deterministic Python establishes the facts, LangGraph coordinates the workflow, Mem0 supplies historical context, validation protects accuracy, and the AI layer turns verified evidence into concise executive insight.

20. References

Project repository: <https://github.com/svedamur22-2ai/elt-portfolio-agent>

Architecture visual reference: https://chatgpt.com/s/m_6a9a16c9fd408191a46dc402323cda20

Primary repository references reviewed: README.md, architecture.md, requirements.txt, src/, app/, notebooks/ and docs/sample_expected_elt_report.md.